Lec12 Backprop & Improving Neural Network

Recap: Logistic Regression = a one-neuron neural network

= linear part + activation part (sigmoid for classification)

↓ casts a number $(-\infty, \infty)$ into $(0,1)$

↓ which can be interpreted as a probability

More layers in Neural Network → copy more complex data ∴ flexibility

In Lec11, we stopped at a point where we did a forward propagation

- had an example, forward propagation, get the output, compute the cost function which compares this output to the ground truth, and then we were in the process of backpropagating the error to tell our parameters how they should move in order to detect cats more properly

Backpropagation

In order to define our optimization problem & find the right parameters, we need to define

cost function (of a batch of m examples) $J(\hat{y}, y) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}^{(i)}(\hat{y}, y)$

↓ for vectorization, for parallelization of GPU computation

with $\mathcal{L}^{(i)} = -\left[y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)})\right]$

update: $w^{[l]} = w^{[l]} - \alpha \frac{\partial J}{\partial w^{[l]}}$

$b^{[l]} = b^{[l]} - \alpha \frac{\partial J}{\partial b^{[l]}}$

$\frac{\partial J}{\partial w^{[2]}}$ $\frac{\partial \mathcal{L}}{\partial w^{[2]}} = -\left[y^{(i)} \frac{\partial}{\partial w^{[2]}} \log\left(\sigma(w^{[2]} a^{[2]} + b^{[2]})\right) + (1-y^{(i)}) \frac{\partial}{\partial w^{[2]}} \left(\log(1-\sigma(w^{[2]} a^{[2]} + b^{[2]}))\right)\right]$

① $\log' x = \frac{1}{\log x}$

② $\frac{\partial \log \sigma(\cdots)}{\partial w} = \frac{1}{\sigma(\cdots)} \cdot \frac{\partial \sigma(\cdots)}{\partial w^{[2]}}$

③ $\sigma'(x) = \sigma(x)(1-\sigma(x))$

④ $\frac{\partial \sigma(w^{[2]} a^{[2]} + b^{[2]})}{\partial w^{[2]}} = a^{[2]}(1-a^{[2]}) \cdot \frac{\partial}{\partial w^{[2]}}(w^{[2]} \cdot a^{[2]} + b^{[2]})$

$w^{[2]}$ ∈ (1,2) ∴ two neurons connected to one neuron

$w^{[2]} a^{[2]} + b^{[2]}$ = scalar ∴ $a^{[2]}$ 와 $\hat{y}$ 도 scalar 여야 함

$a^{[2]} = (2,1)$ ∴ $a^{[2]T}$ 이어야 (1,2)(2,1) 이 되므로

$= -\left[y^{(i)} \frac{1}{a^{[2]}} \cdot a^{[2]}(1-a^{[2]}) \cdot a^{[2]T} + (1-y^{(i)}) \cdot \frac{1}{1-a^{[2]}} (-1) a^{[2]} \cdot (1-a^{[2]}) \cdot a^{[2]T}\right]$

$= -\left[y^{(i)}(1-a^{[2]}) a^{[2]T} - (1-y^{(i)}) \cdot a^{[2]} \cdot a^{[2]T}\right]$

$= -\left[y^{(i)} a^{[2]T} - a^{[2]} a^{[2]T}\right]$

$= -(y^{(i)} - a^{[2]}) a^{[2]T}$ → $\frac{\partial z^{[2]}}{\partial w^{[2]}}$

$\because z^{[2]} = w^{[2]} a^{[2]} + b^{[2]}$

$\frac{\partial J}{\partial w^{[2]}} = -\frac{1}{m}\sum_{i=1}^{m} (y^{(i)} - a^{[2]}) a^{[2]T}$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial W^{[1]}}$$

(2,3)

$(a^{[2]} - y)$ $\quad W^{[2]T}$ $\quad a^{[1]}(1-a^{[1]})$ $\quad a^{[0]T}$

∵ 3 neurons to 2 neurons

$$= (a^{[2]} - y) \cdot W^{[2]T} * a^{[1]} \cdot (1 - a^{[1]})\, a^{[0]T}$$

(1,1) (2,1) (2,1) (1,3)

element-wise product

↓ shape shift to fit to $\frac{\partial \mathcal{L}}{\partial W^{[1]}}$ shape (2,3)

$$= W^{[2]T} * a^{[1]}(1-a^{[1]}) \cdot (a^{[2]} - y) \cdot a^{[0]T}$$

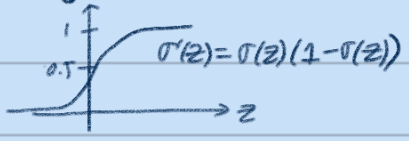
(2,1) (1,1) (1,3)

$$\frac{\partial J}{\partial W^{[1]}} = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial \mathcal{L}}{\partial W^{[1]}}$$ use this to update $W^{[1]}$

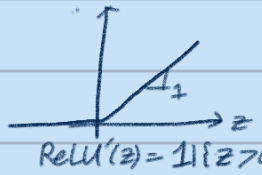
Improving your neuron networks

1. Try different activation functions

ex1) sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$



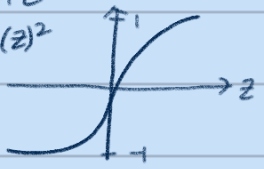
ReLU$(z) = \begin{cases} 0; & z \le 0 \\ z; & z > 0 \end{cases}$



ReLU$'(z) = 1\{z > 0\}$

$\tan h(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

$\tan h'(z) = 1 - \tan h(z)^2$

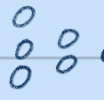


(가) we use it for classification

(나) 값이 너무 크거나 작으면 gradient가 너무 작아져 backprop할 때 vanish → hard to update params at the early layer — saturated activation issue

(가) (나) sigmoid와 비슷

why do we need activation function?



activations = Id $z \to z$

$$\hat{y} = a^{[2]} = z^{[2]} = W^{[2]} \cdot a^{[1]} + b^{[2]}$$

$$= W^{[2]}(W^{[1]}a^{[1]} + b^{[1]}) + b^{[2]}$$

$$= W^{[2]}W^{[1]}(W^{[1]}x + b^{[1]}) + W^{[2]} \cdot b^{[1]} + b^{[2]}$$

$$= Wx + b$$

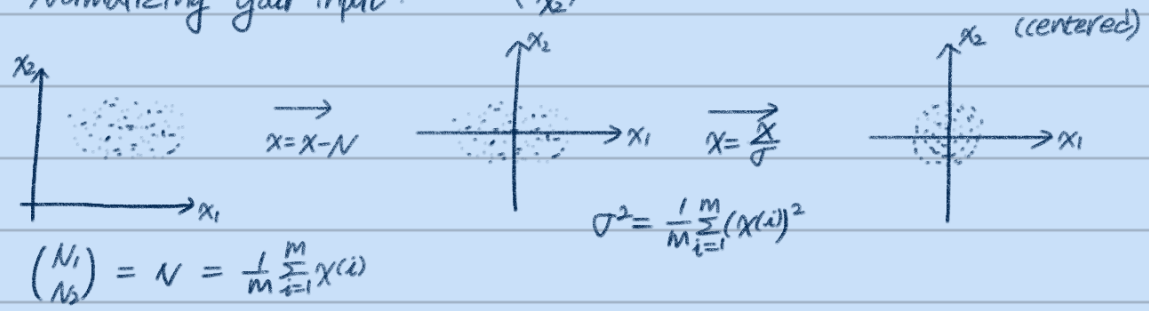
with $W = W^{[2]}W^{[1]}W^{[1]}$

$B = W^{[2]}W^{[1]} \cdot b + W^{[2]}b^{[1]} + b^{[2]}$

⇒ Without activation functions, no matter how deep the neural net is, it's gonna be eq to linear regression

2. Initialization methods

Normalizing your input: $x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$



$\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} = \mu = \frac{1}{m}\sum_{i=1}^{m} x^{(i)}$

$\sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)})^2$

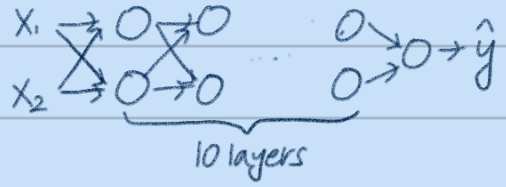
① why centered data is better to train?

loss function:

Apparently GD finds the steepest slope and with the centered data (right side), the steeper slope always pointing towards the middle

② the μ & σ should be earned from the training set and the same μ & σ should be used for test set

Vanishing / exploding gradients



$g: z \mapsto z$

$b = 0$

∵ activation is an identity function

$\hat{y} = w^{[L]} \cdot a^{[L-1]} = w^{[L]} \cdot w^{[L-1]} \cdot a^{[L-2]}$

$= w^{[L]} \cdot w^{[L-1]}, \cdots \cdot w^{[1]} \cdot x$

$w^{[L]} = \begin{pmatrix} 1.5 & 0 \\ 0 & 1.5 \end{pmatrix} \rightarrow \begin{pmatrix} 1.5^L & 0 \\ 0 & 1.5^L \end{pmatrix}$

Example with 1 neuron:

$a = \sigma(z)$

$z = w_1 x_1 + \cdots + w_n x_n$

larger n → small w

$w_i \sim \frac{1}{n}$

$w^{[L]} = $ np.random.randn(shape) * np.sqrt($\frac{2}{n^{[L-1]}}$)

for sigmoid

2 for ReLU

Xavier Initialization $w^{[L]} \sim \sqrt{\frac{1}{n^{[L-1]}}}$ for tanh

He Initialization $w^{[L]} \sim \sqrt{\frac{2}{n^{[L]} + n^{[L-1]}}}$

3. Optimization

mini batch gradient descent (stochacity <> batch ~101)

$X = (x^{(1)}, x^{(2)}, \cdots, x^{(m)})$

$Y = (y^{(1)}, y^{(2)}, \cdots, y^{(m)})$

$X = (x^{\{1\}}, \cdots, x^{\{T\}})$ curly braket: batch $x^{\{1\}} = (x^{(1)}, x^{(2)} \cdots x^{(1000)})$

$Y = (y^{\{1\}}, \cdots, y^{\{T\}})$

Algo: For iteration $t=1 \cdots$

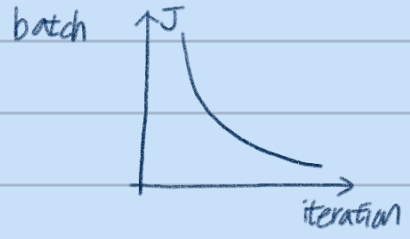
Select batch $(x^{\{t\}}, y^{\{t\}})$

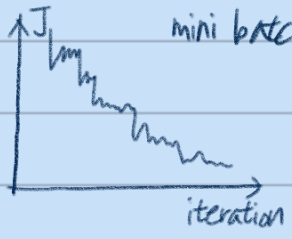
Foward Propagation $J = \frac{1}{1000} \sum_{i=1}^{1000} \mathcal{L}(i)$

Backprop Batch

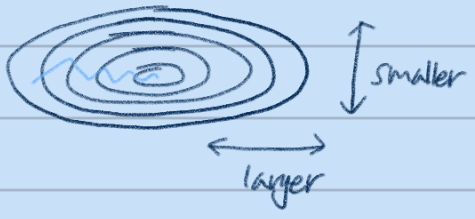
Update

cost function graph:





GD + Momentum Algorithm



avg of prev velocity

$V = \beta V + (1-\beta)\frac{\partial \mathcal{L}}{\partial W}$ tracks the direction

$W = W + \alpha \cdot V$